The methods `to_date`, `to_time`, and `to_datetime` are basically convenience wrappers around `Date._parse`:

```
"2010-07-27".to_date              # => Tue, 27 Jul 2010
"2010-07-27 23:37:00".to_time     # => Tue Jul 27 23:37:00 UTC 2010
"2010-07-27 23:37:00".to_datetime # => Tue, 27 Jul 2010 23:37:00 +0000
```

`to_time` receives an optional argument `:utc` or `:local`, to indicate which time zone you want the time in:

```
"2010-07-27 23:42:00".to_time(:utc)   # => Tue Jul 27 23:42:00 UTC
2010
"2010-07-27 23:42:00".to_time(:local) # => Tue Jul 27 23:42:00 +0200
2010
```

Default is `:utc`.

Please refer to the documentation of `Date._parse` for further details.

The three of them return `nil` for blank receivers.

Defined in `active_support/core_ext/string/conversions.rb`.

6 Extensions to `Numeric`

6.1 Bytes

All numbers respond to these methods:

```
bytes
kilobytes
megabytes
gigabytes
terabytes
petabytes
exabytes
```

They return the corresponding amount of bytes, using a conversion factor of 1024:

```
2.kilobytes   # => 2048
3.megabytes   # => 3145728
3.5.gigabytes # => 3758096384
-4.exabytes   # => -4611686018427387904
```

Singular forms are aliased so you are able to say:

```
1.megabyte # => 1048576
```

Defined in `active_support/core_ext/numeric/bytes.rb`.

6.2 Time

Enables the use of time calculations and declarations, like `45.minutes + 2.hours + 4.years`.

These methods use Time#advance for precise date calculations when using from_now, ago, etc. as well as adding or subtracting their results from a Time object. For example:

```
# equivalent to Time.current.advance(months: 1)
1.month.from_now

# equivalent to Time.current.advance(years: 2)
2.years.from_now

# equivalent to Time.current.advance(months: 4, years: 5)
(4.months + 5.years).from_now
```